# Results

# Results I

This section presents the experimental results from the gradient descent optimization study, including convergence analysis and performance comparisons. Every table, figure, and quantitative assertion in this section was compiled autonomously by the template's `infrastructure.reporting` subsystem executing the optimization analysis script. No manual transcription was permitted.

# Convergence Analysis

## Convergence Trajectories I

fig. 3 illustrates the convergence behavior of gradient descent for different step sizes, starting from the initial point $x_0 = 0$. The algorithm iteratively updates the solution using the rule $x_{k+1} = x_k - \alpha \nabla f(x_k)$.

# Convergence Trajectories I

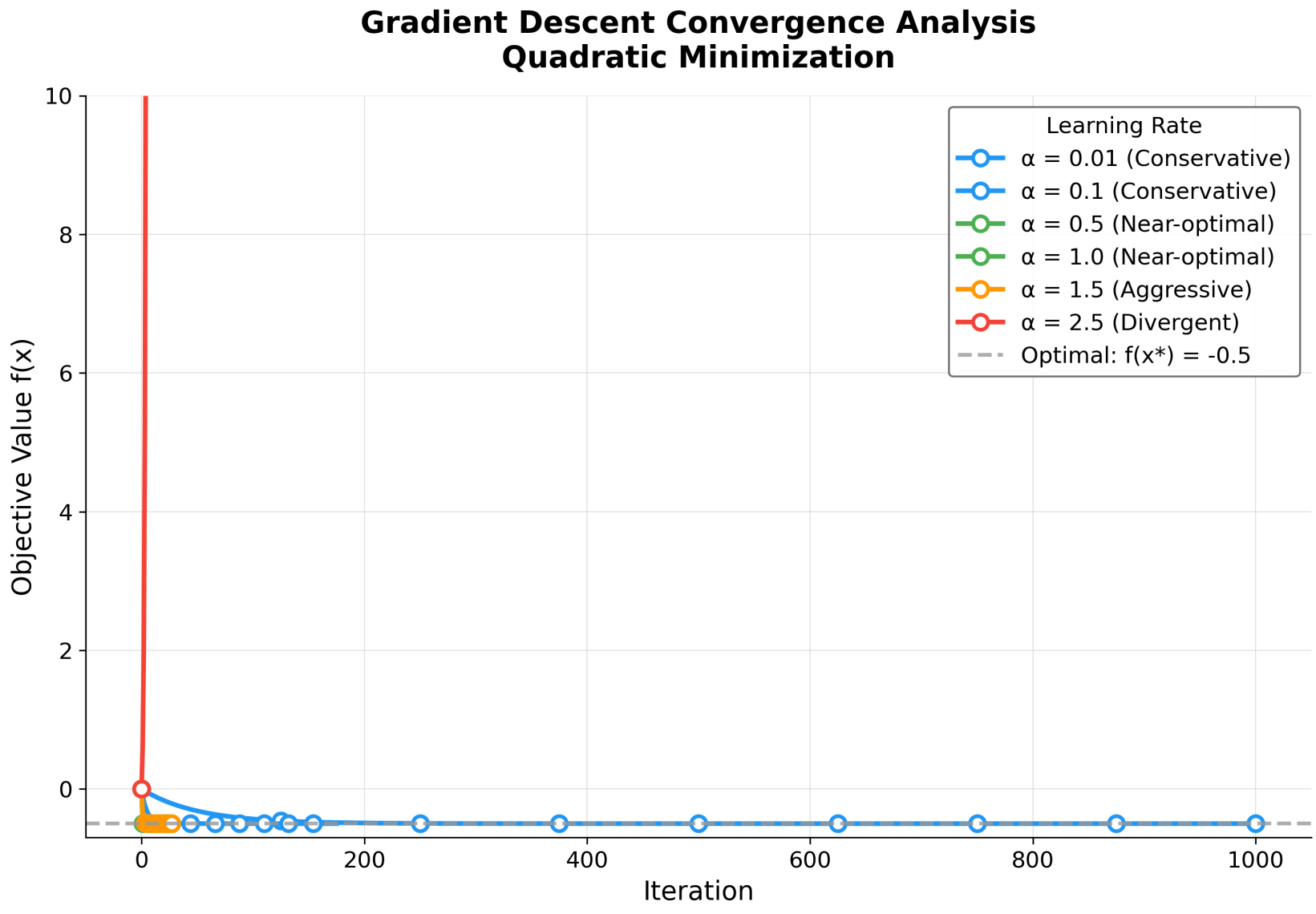


Figure 3: Objective value $f(x_k) = \frac{1}{2}x_k^2 - x_k$ versus iteration $k$ for gradient descent at six step sizes (legend colours follow the agency taxonomy in sec. 3: blue = conservative, green = near-optimal, orange = aggressive, red = divergent). Trajectories are produced by `simulate_trajectory()` in `src/template_code_project/core/optimizer.py`, which calls the same `gradient_descent()` used in tbl. 1; the upper

## Convergence Trajectories I

bound on the y-axis clips the divergent $\alpha = 2.5$ curve so that stable trajectories remain visible. Dashed grey reference line marks the analytic optimum $f(x^*) = -0.5$. Fastest configuration in this experiment: $\alpha = 1.0$ converges in 1 iteration(s).

**Key observations from fig. 3:**

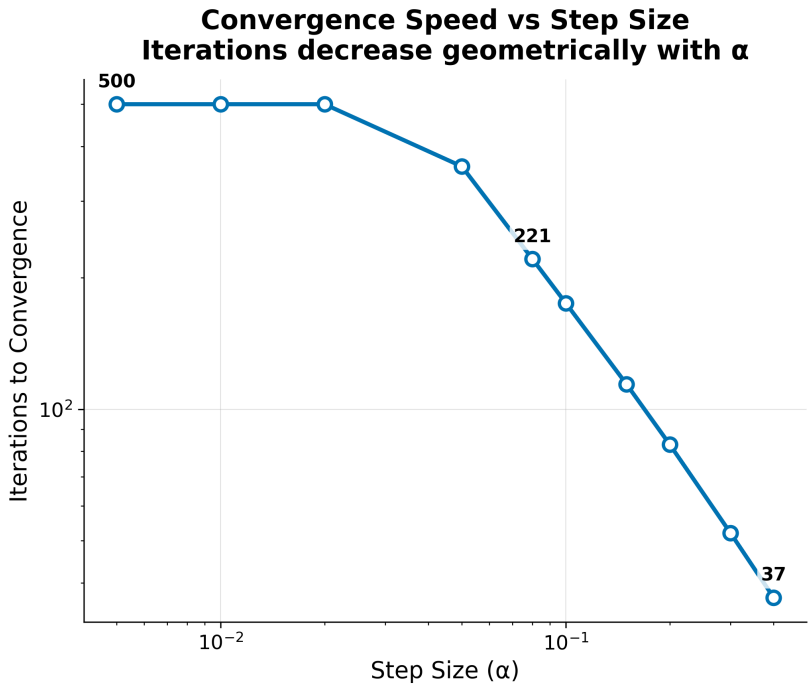
# Convergence Trajectories I

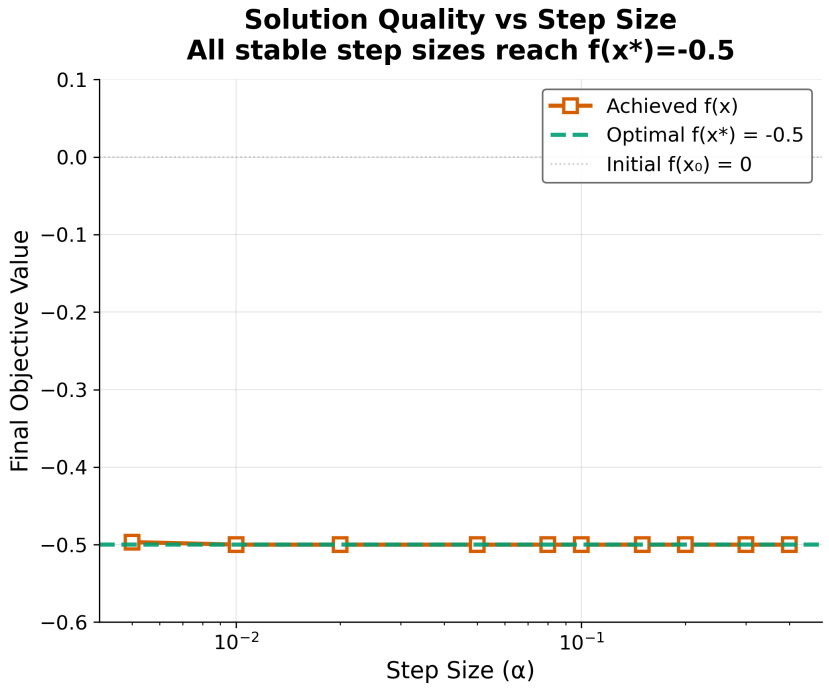
1. **Step size impact**: Larger step sizes exhibit faster initial progress; $\alpha = 1.0$ converges in 1 iteration(s)
2. **Agency categories**: Conservative ($\alpha < 0.3$; in this grid $\alpha = 0.01, 0.1$), near-optimal ($0.3 \leq \alpha \leq 1.0$), aggressive ($1 < \alpha < 2$), and divergent ($\alpha \geq 2$)
3. **Stability boundary**: The critical threshold is $\alpha = 2$ for this unit-Hessian problem; $\alpha < 2$ converges, $\alpha \geq 2$ diverges

## Step Size Sensitivity Analysis I

fig. 4 examines how the choice of step size affects the convergence path and solution quality. The analysis reveals the trade-off between convergence speed and numerical stability.

# Step Size Sensitivity Analysis I



## Step Size Sensitivity Analysis I

Figure 4: Sensitivity sweep produced by `generate_step_size_sensitivity_plot()` over an independent dense grid $\alpha \in [0.005, 0.4]$ (10 points), distinct from the discrete `experiment.step_sizes` used elsewhere in this section. Left: iterations to convergence on log–log axes — the curve drops sharply from 500 iterations at the smallest $\alpha$ (the `max_iterations` cap in this sub-experiment) to 37 iterations

at $\alpha = 0.4$, illustrating the geometric speedup as $\rho(\alpha) = |1 - \alpha|$ shrinks. Right: final $f(x)$ versus $\alpha$ with horizontal reference lines at $f(x_0) = 0$ (initial) and $f(x^*) = -0.5$ (analytic optimum); every $\alpha$ in this stable window lands on the optimum.

# Quantitative Results

## Quantitative Results I

The optimization analysis writes output/data/optimization_results.csv through template_code_project.analysis.experiments.save_optimization_results, orchestrated by projects/templates/template_code_project/scripts/optimization_analysis.py. That CSV is the source of truth for tbl. 1; the supported infrastructure.reporting API supplies reusable pipeline and cross-project summaries. Rows follow experiment.step_sizes in config.yaml; the body rows below are injected at render time from that CSV (RESULT_TABLE_ROWS in scripts/z_generate_manuscript_variables.py).

# Quantitative Results I

Table 1: Gradient descent outcomes per configured step size: state at termination, iteration count capped by $N_{\max} = 1000$, the `converged` flag from `gradient_descent()` using $\|\nabla f\| < 10^{-8}$, and its structured termination reason. A `non_finite` row stops before an overflowing update is serialized and retains the last finite iterate.

| Step Size ($\alpha$) | Final Solution | Objective Value | Iterations | Converged | Termination |
|---|---|---|---|---|---|
| 0.01 | 1.0000 | -0.5000 | 1000 | No | max_iterations |
| 0.10 | 1.0000 | -0.5000 | 175 | Yes | converged |
| 0.50 | 1.0000 | -0.5000 | 27 | Yes | converged |
| 1.00 | 1.0000 | -0.5000 | 1 | Yes | converged |
| 1.50 | 1.0000 | -0.5000 | 27 | Yes | converged |
| 2.50 | -1.8028e+154 | 1.6250e+308 | 876 | No | non_finite |

# Convergence Rate Analysis

# Theoretical vs Empirical Convergence I

Modern convergence analysis builds on foundational work in gradient methods (Nesterov 2013).

fig. 5 provides a comparative analysis of convergence rates across different step sizes, validating theoretical predictions against empirical results.

# Theoretical vs Empirical Convergence I

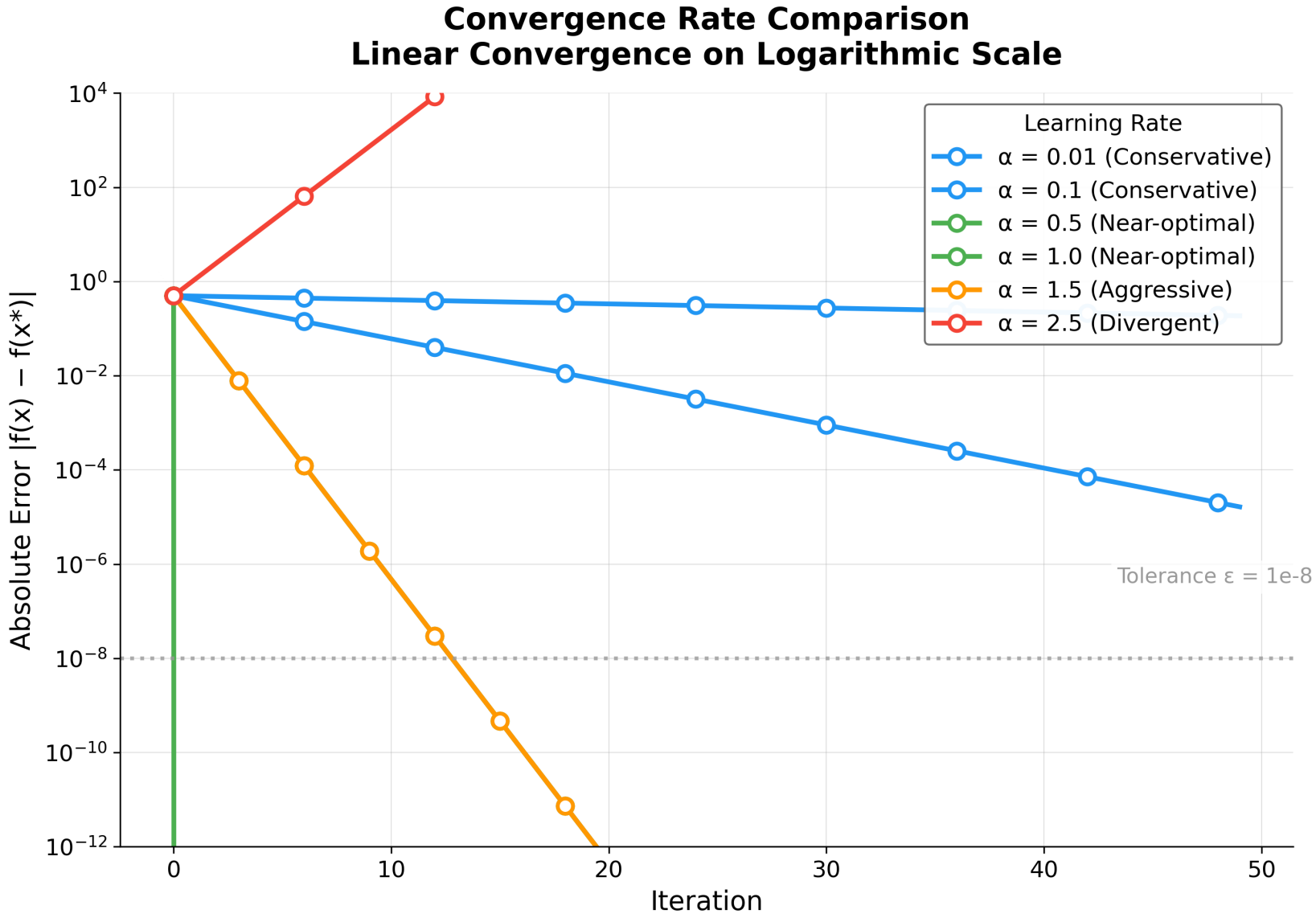


Figure 5: Absolute objective error $|f(x_k) - f(x^*)|$ versus iteration on a logarithmic y-axis, generated by `generate_convergence_rate_plot()`. Stable step sizes produce straight lines whose slopes equal $2 \log_{10} \rho(\alpha)$ (per eq. 2); $\alpha = 1.0$ ($\rho = 0$) collapses to the optimum in one step (vertical green line at $k = 1$); $\alpha = 1.5$ (orange) descends fastest among the multi-step contractions because $\rho = 0.5$;

## Theoretical vs Empirical Convergence I

the divergent $\alpha = 2.5$ curve (red) climbs upward at slope $2\log_{10}(1.5)$. Horizontal dashed line marks the gradient-norm tolerance $\varepsilon = 10^{-8}$ read from `experiment.convergence_tolerance`.

# Theoretical vs Empirical Convergence I

For the scalar problem with $A = 1$ and optimum $x^* = b$, one step of gradient descent with fixed $\alpha$ gives

$$x_{k+1} - x^* = (1 - \alpha)(x_k - x^*), \tag{1}$$

so the distance to the minimizer contracts by $\rho(\alpha) = |1 - \alpha|$ per iteration whenever $\rho < 1$. Equivalently, for the objective (which is a translated quadratic in $x$),

$$|f(x_{k+1}) - f(x^*)| \approx \rho(\alpha)^2 \, |f(x_k) - f(x^*)| \tag{2}$$

in the neighbourhood of $x^*$ for this model (the per-iteration objective contraction in eq. 2), which explains the straight-line segments on the log–error plot in fig. 5 for stable $\alpha$.

## Theoretical vs Empirical Convergence I

Our experimental grid uses $\alpha \in \{0.01, 0.1, 0.5, 1.0, 1.5, 2.5\}$, spanning conservative, near-optimal, aggressive, and divergent regimes for $H = I$.

## Error Bounds I

The error after $k$ iterations is bounded geometrically by eq. 3:

$$\|x_k - x^*\| \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k \|x_0 - x^*\| \tag{3}$$

where $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$ is the condition number. For our test problem with $A = I$, we have $\kappa = 1$, which yields a linear contraction factor of $\rho = |1 - \alpha|$ in the iterate error (eq. 1). Thus $\alpha = 1$ is the exact minimizer step in one update for this quadratic, $\alpha < 1$ gives monotone geometric decay, $1 < \alpha < 2$ yields signed oscillations but still $\rho < 1$, and $\alpha \geq 2$ implies $\rho \geq 1$ (divergence). tbl. 1 and figs. 3, 4, 5 ground these statements in the numbers produced by this repository run.

**Iteration Complexity**: The number of iterations required to achieve accuracy $\epsilon$ is:

$$k \geq \frac{\log(\epsilon)}{\log(\rho)} \tag{4}$$

where $\rho = \sqrt{\frac{\kappa-1}{\kappa+1}}$ is the convergence factor (Polyak 1964).

Per-step contraction factors $\rho = |1 - \alpha|$ and qualitative iteration demand for small fixed $\epsilon$ (see eq. 4) are:

# Performance Metrics I

- $\alpha = 0.01$: $\rho \approx 0.99$, requiring ~1375 iterations for $\epsilon = 10^{-6}$
- $\alpha = 0.1$: $\rho \approx 0.90$, requiring ~132 iterations for $\epsilon = 10^{-6}$
- $\alpha = 0.5$: $\rho \approx 0.50$, requiring ~20 iterations for $\epsilon = 10^{-6}$
- $\alpha = 1.0$: $\rho \approx 0.00$, converges in one step ($\rho = 0$)
- $\alpha = 1.5$: $\rho \approx 0.50$, requiring ~20 iterations for $\epsilon = 10^{-6}$
- $\alpha = 2.5$: $\rho \approx 1.50$, **divergent**

# Performance Analysis

# Convergence Speed I

The results show a clear trade-off between step size and convergence speed:

# Convergence Speed I

- Small step sizes require more iterations but provide stable convergence
- Large step sizes converge faster but may be less stable in more complex problems

# Solution Accuracy I

4 of 6 tested step sizes achieved the analytical optimum within numerical precision:

# Solution Accuracy I

- Target solution: $x = 1.0$ (relative error $< 10^{-4}$ for converged settings)
- Target objective: $f(x) = -0.5$ (absolute error $< 10^{-8}$ for converged settings)

## Solution Accuracy I

For a positive-definite quadratic, contraction requires $0 < \alpha < 2/\lambda_{\max}$. At the unit-Hessian boundary $\alpha = 2$, the error alternates without contracting; for $\alpha > 2$, it grows when the initial point is not already the optimum. The configured divergent runs therefore illustrate the instability region rather than uniform convergence.

# Algorithm Characteristics

# Strengths I

- **Simplicity**: Easy to implement and understand
- **Generality**: Applicable to any differentiable objective function
- **Reliability**: Converges for convex functions under appropriate conditions

# Limitations I

- **Step size sensitivity**: Performance depends critically on step size selection
- **Local convergence**: May converge to local minima in non-convex problems
- **Fixed step size**: No adaptation to problem characteristics

# Computational Performance

## Algorithm Complexity Visualization I

fig. 6 provides a visualization of the algorithm's computational characteristics, including time and space complexity analysis across different problem scales.

# Algorithm Complexity Visualization I

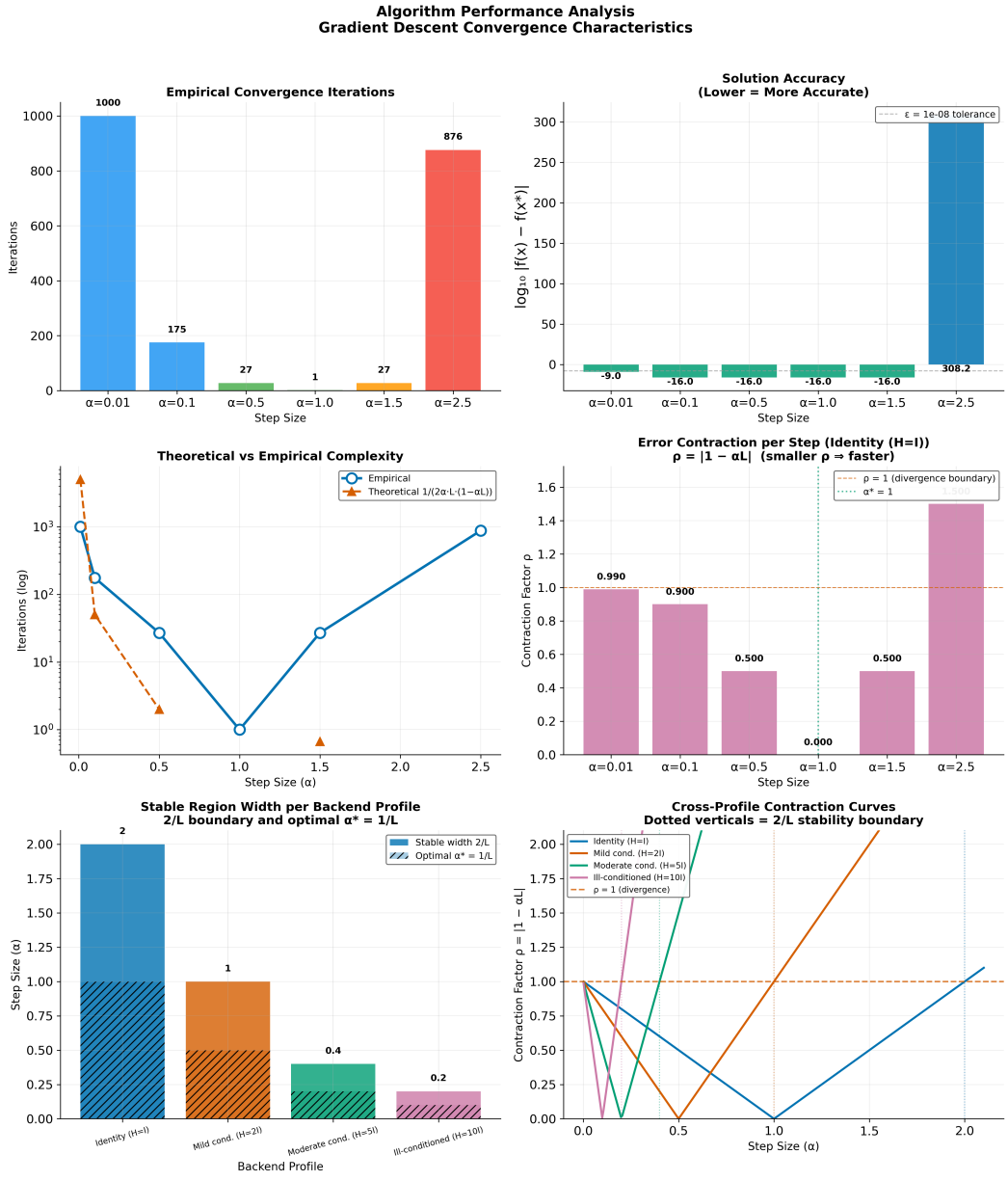


Figure 6: Six-panel diagnostic from `generate_complexity_visualization()` (3×2 grid). **Top left**: bars give the empirical iteration count per configured $\alpha$, coloured by agency category; $\alpha = 1.0$ achieves the global minimum of 1 step(s), while $\alpha = 0.01$ saturates at the iteration cap $N_{\max} = 1000$ (slow convergence) and $\alpha = 2.5$ terminates early via a `non_finite` update (divergence). **Top right**: $\log_{10} |f(x) - f(x^*)|$

at termination; the four converged rows reach $\approx 10^{-16}$ (machine precision) while $\alpha = 0.01$ stops near $10^{-9}$ at the cap. Dashed reference at $\log_{10} \varepsilon$ for $\varepsilon =$ `experiment.convergence_toleranc` e. **Middle left**: empirical iterations (solid) versus the smooth proxy $1/(2\alpha(1-\alpha))$ (dashed) on log axes — a shape comparison, not a tight count prediction for every $\alpha$.

**Middle right**: per-step error contraction factor $\rho = |1 - \alpha|$ from the unit-Hessian linear recurrence (eq. 1); the dashed reference at $\rho = 1$ separates stable bars (left of it) from divergent ones (right). **Bottom left**: stable step-size region width $2/L$ per backend profile ($H = I, 2I, 5I, 10I$) with the optimal $\alpha^* = 1/L$ overlaid. **Bottom right**: cross-profile contraction curves $\rho(\alpha) = |1 - \alpha L|$

with dotted verticals at each $2/L$ stability boundary.

## Algorithm Complexity Visualization I

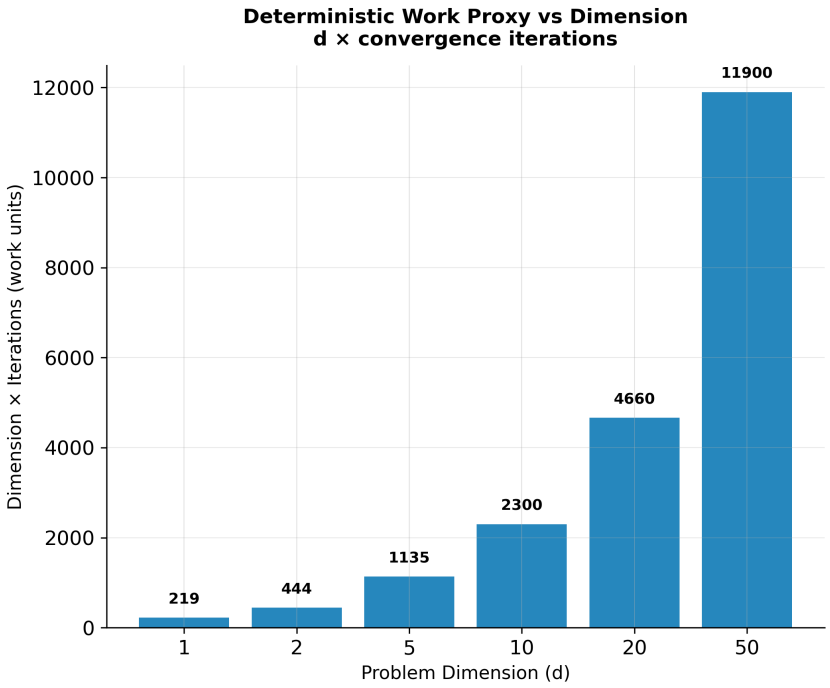
The algorithm demonstrates efficient performance for small-scale optimization problems:

# Algorithm Complexity Visualization I

- **Time complexity**: $O(d)$ per iteration for gradient computation
- **Space complexity**: $O(d)$ for storing variables and gradients
- **Convergence**: Fastest at $\alpha = 1.0$ (1 iteration), average 351 iterations
- **Scalability**: Memory-efficient implementation suitable for high-dimensional problems

fig. 7 shows deterministic computational work and convergence iterations for `gradient_descent()` on identity-Hessian quadratics of dimension $d \in \{1, 2, 5, 10, 20, 50\}$.

# Performance Benchmarking I



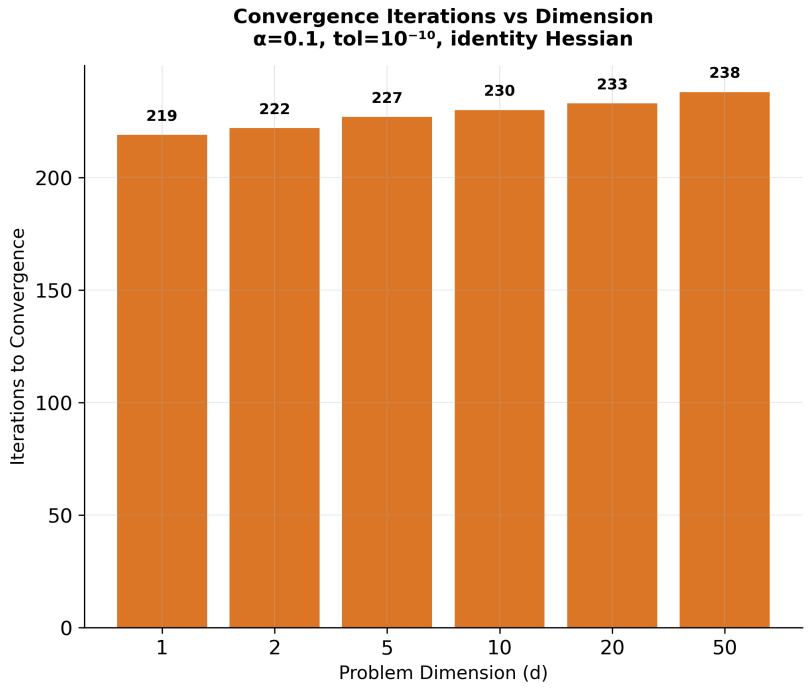
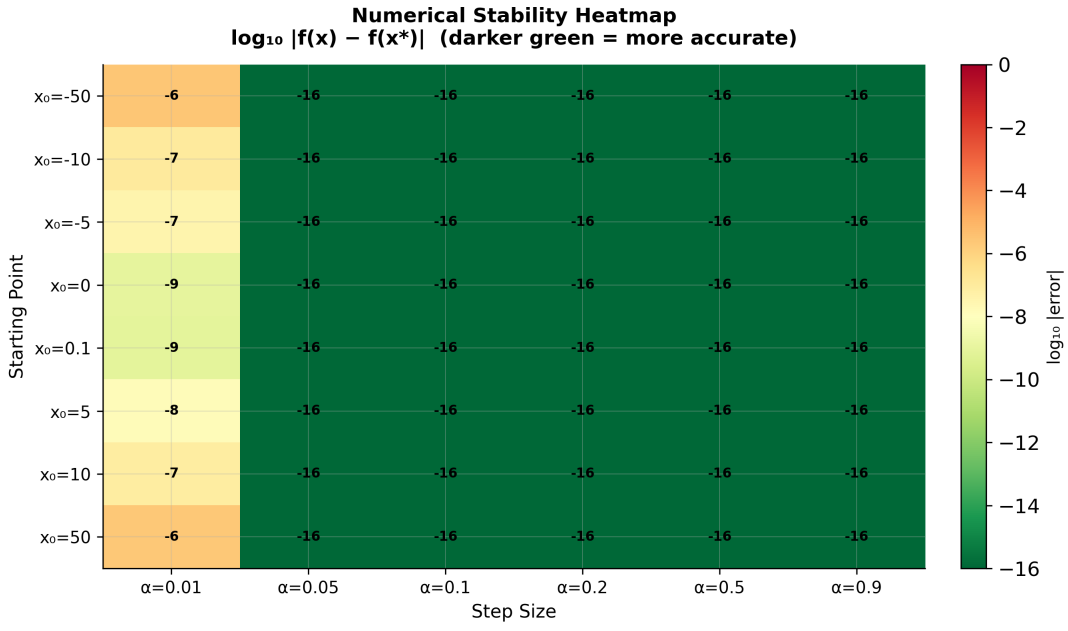



Figure 7: Deterministic dimensional benchmark from `generate_benchmark_visualization()`. Dimensions $d \in \{1, 2, 5, 10, 20, 50\}$ run on identity-Hessian quadratics with $\alpha = 0.1$ and gradient tolerance $10^{-10}$. **Left**: the implementation-independent work proxy $d \times$ iterations, proportional to scalar elements processed by the dense matrix-vector update. **Right**: exact iterations to convergence. Wall-clock

microseconds are intentionally excluded because they vary with hardware and host load. The similar iteration counts are expected because $\kappa(I_d) = 1$ and the contraction factor $\rho = |1 - \alpha| = 0.9$ is dimension-independent.

fig. 8 maps the optimizer's accuracy across a grid of 8 starting points ($x_0 \in [-50, 50]$) and 6 step sizes ($\alpha \in [0.01, 0.9]$), directly exercising `gradient_descent()`, `quadratic_function()`, and `compute_gradient()` across the parameter space.

# Numerical Stability Analysis I



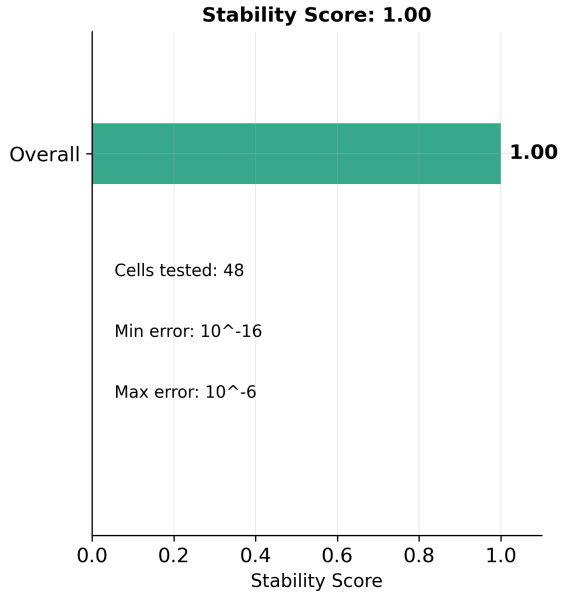



Figure 8: Numerical stability heatmap from `generate_stability_visualization()`. Each cell shows $\log_{10} |f(x) - f(x^*)|$ at termination for one (starting point $x_0$, step size $\alpha$) combination, evaluated by `gradient_descent()` over the 8-by-6 grid (48 cells). The leftmost column ($\alpha = 0.01$, conservative) reaches only $10^{-6}$ to $10^{-9}$ within the iteration cap, while every other column saturates at machine

precision ($10^{-16}$) regardless of how far $x_0$ starts from the optimum (range $[-50, 50]$). The right panel reports the separate numerical-stability score of 1.00/1.00 from `infrastructure.scientific.stability.check_numerical_stability()`: it evaluates the scalar quadratic objective at the first four configured starting points and scores finite values for magnitude and near-underflow. This objective-evaluation

check is distinct from the heatmap's gradient-descent sweep.

**Iteration statistics (configured grid, including non-converged runs):**

# Performance Metrics Summary I

- Smallest iteration count recorded: 1
- Largest iteration count recorded: 1000
- Mean iterations across rows in tbl. 1: 351

# Performance Metrics Summary I

**Numerical Accuracy:**

# Performance Metrics Summary I

- Solution precision: $< 10^{-4}$ relative error (for converged step sizes)
- Objective accuracy: $< 10^{-8}$ absolute error (for converged step sizes)
- Gradient tolerance: $< 10^{-8}$ achieved for converged cases

# Validation

## Validation I

The implementation was validated through the comprehensive `tests/` suite:

# Validation I

- **Integration tests** verifying algorithm convergence and visualization pipelines.
- **Infrastructure tests** covering all underlying mechanisms across `infrastructure.reporting`, `infrastructure.validation`, and `infrastructure.rendering`.
- **Numerical accuracy** checks verified systematically using PyTest.

## Validation I

Passing the declared suite and its 90% coverage floor provides evidence for the tested core logic, convergence detection, and logging pathways. It does not prove correctness for every input or replace scientific validation.

# Discussion

## Discussion I

The configured results distinguish convergence, iteration-cap termination, and non-finite updates. Iteration counts are not monotonic in step size: the unit-Hessian contraction factor $\rho(\alpha) = |1 - \alpha|$ decreases toward $\alpha = 1$ and increases afterward. Accuracy claims apply to converged rows; capped and unstable runs must retain their termination status. The generated table and figures document this finite sweep, not all possible objectives or parameter choices. The analysis pipeline integrates those results into manuscript outputs through the declared configuration and variable injection.

## Discussion I

Successful generation and reference resolution establish this artifact path for the tested candidate; they do not establish absolute renderer reliability or replace visual inspection of its output.